

REBUILDING YOUR TCP CHAT

C# to TypeScript / Node.js

One host + two authenticated users

Goal: Recreate the architecture you already built in C#: TCP server, multiple clients, Login/Register, Users.json persistence, PBKDF2 password hashing, JSON messages, Chat/Join/Leave events, broadcasting and graceful /quit - but implement it with TypeScript and Node.js.

What this guide assumes you already understand from C#: classes, async/await, DTOs, JSON, TCP basics, authentication flow, collections, file IO and separation of responsibilities. The goal is not to relearn networking from zero; it is to translate your mental model into TypeScript correctly.

Your C# project	TypeScript recreation
ChatServer	ChatServer using node:net Server
ChatClient	ChatClient using node:net Socket
ClientConnection	Class wrapping Socket + JSON-line framing
MessageDispatcher	Broadcast service over Set<ClientConnection>
AuthenticationService	Login/Register orchestration
PasswordHasher	node:crypto PBKDF2 + timingSafeEqual
Users.json	fs/promises JSON persistence
Task / async Task	Promise / async function

Recommended working style: open three terminals - one server terminal and two client terminals. Build one vertical slice at a time and run npm run check after every step.

1. Architecture First

Keep the C# responsibilities; change the runtime and APIs.

Target runtime flow



The host is the server process. It does not need to be a third chatting user. It owns the listening socket and the connected-client collection. Alice and Bob are separate client processes connecting to the host.

Core rule: Do not put authentication, broadcasting, socket parsing, file persistence and console UI into one TypeScript file. Node makes it easy to write everything inline; resist that. Preserve the class boundaries that made your C# project understandable.

Project structure

```

tcp-chat-ts/
package.json
tsconfig.json
data/
  users.json
src/
  shared/
    messages.ts
    jsonLines.ts
  server/
    main.ts
    chatServer.ts
    clientConnection.ts
    messageDispatcher.ts
    authenticationService.ts
    passwordHasher.ts
    userStore.ts
  client/
    main.ts
    chatClient.ts
    consoleInput.ts

```

Why one repository instead of two projects? For this learning version, server and client can share the same TypeScript message types without publishing a package. You still run them as separate Node processes. Later, you can split them into workspaces/packages if you want to practise monorepo architecture.

2. C# -> TypeScript Conversion Map

C#	TypeScript / Node	Important difference
TcpListener	<code>`net.Server`</code> from <code>`node:net`</code>	Listens for TCP connections.
AcceptTcpClientAsync()	<code>`net.createServer(socket => ...)`</code> / connection event	Node is event-driven; callback receives the connected Socket.
TcpClient	<code>`net.Socket`</code>	The socket represents the TCP connection.
NetworkStream	<code>`net.Socket`</code> itself	Socket is a Duplex stream: readable + writable.

StreamReader.ReadLineAsync()	custom newline parser over Socket chunks	TCP delivers bytes/chunks, not messages.
StreamWriter.WriteLineAsync()	<code>`socket.write(text + "\n")`</code>	The newline is your application framing delimiter.
Task	<code>`Promise<T>`</code>	Represents asynchronous completion.
async Task Foo()	<code>`async function foo(): Promise<void>`</code>	Same high-level async idea.
Task.WhenAny()	<code>`Promise.race()`</code>	Useful conceptually for competing async operations.
List<ClientConnection>	<code>`Set<ClientConnection>`</code>	A Set is convenient for add/delete connected sessions.
class DTO / record	<code>`interface`</code> or <code>`type`</code>	Compile-time shape; disappears at runtime.
enum MessageType	string-literal union	<code>`"Chat"   "Join"   "Leave"`</code> is idiomatic and serializes cleanly.
JsonSerializer.Serialize	<code>`JSON.stringify()`</code>	Object -> JSON text.
JsonSerializer.Deserialize<T>	<code>`JSON.parse()`</code> + validation/narrowing	JSON.parse is runtime-unsafe unless validated.
File.ReadAllTextAsync	<code>`readFile()`</code> from <code>`node:fs/promises`</code>	Async file IO.
CancellationToken	<code>`AbortSignal`</code>	Closest common cancellation mechanism.

Hidden mechanic: TypeScript types do not exist at runtime. ``JSON.parse()`` can return any shape. Your C# ``Deserialize<T>()`` mental model is stricter than plain JavaScript. For a training project, a discriminant field such as ``type`` plus explicit checks is enough; production systems often use a runtime validator such as Zod or Valibot.

3. Create the TypeScript Project

1. Create a new folder named tcp-chat-ts and open a terminal in it.
2. Initialize npm.
3. Install TypeScript, tsx and Node type declarations.
4. Create src/shared, src/server, src/client and data folders.
5. Enable strict TypeScript checking before writing networking code.

Terminal

```
npm init -y
npm install --save-dev typescript tsx @types/node
npx tsc --init
```

package.json - scripts and ESM setup

```
{
  "name": "tcp-chat-ts",
  "private": true,
  "type": "module",
  "scripts": {
    "server": "tsx src/server/main.ts",
    "client": "tsx src/client/main.ts",
    "check": "tsc --noEmit"
  },
  "devDependencies": {
    "@types/node": "*",
    "tsx": "*",
    "typescript": "*"
  }
}
```

```
}
```

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "rootDir": "src",
    "outDir": "dist",
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,
    "esModuleInterop": true,
    "skipLibCheck": true
  },
  "include": ["src/**/*.ts"]
}
```

Why `strict: true`? A socket program has many values that can be absent: `userName` before authentication, parsed fields, disconnected sockets and failed lookups. `strictNullChecks` and the rest of strict mode force you to model those states rather than assuming they exist.

Node ESM detail: With `module: NodeNext`, local imports in TypeScript should normally use the final `.js` extension, for example `import { ChatMessage } from "../shared/messages.js";`. TypeScript resolves that to the `.ts` source while development tools compile/run it as ESM.

4. Translate Your DTOs into TypeScript Types

```
// src/shared/messages.ts

export type AuthType = "Login" | "Register";
export type ChatMessageType = "Chat" | "Join" | "Leave";

export interface AuthRequest {
  type: AuthType;
  userName: string;
  password: string;
}

export interface AuthResponse {
  type: "AuthResponse";
  success: boolean;
  message: string;
}

export interface ChatSendRequest {
  type: "Chat";
  text: string;
}

export interface ChatMessage {
  type: ChatMessageType;
  name: string;
  text: string;
  sentAt: string; // ISO-8601 UTC
}

export type ClientMessage = AuthRequest | ChatSendRequest;
export type ServerMessage = AuthResponse | ChatMessage;
```

C# mental model	TypeScript version
-----------------	--------------------

public sealed class LoginRequest	interface AuthRequest
public string UserName { get; init; }	userName: string
MessageType enum	string-literal union
DateTime SentAt	string containing an ISO timestamp across the wire
base DTO / separate DTO classes	discriminated union keyed by `type`

Why send the timestamp as a string? JSON has no Date type. In C#, System.Text.Json converts DateTime to text. TypeScript should make that wire-format reality explicit. Create it with `new Date().toISOString()` and display it with `new Date(message.sentAt).toLocaleTimeString()`.

5. The Most Important Networking Difference: TCP Has No Message Boundaries

Your C# version used `StreamReader.ReadLineAsync()`. That quietly solved framing for you. Node `Socket` gives chunks of bytes. One `data`/read chunk might contain half a JSON object, one full object, or several objects. Never assume one socket chunk equals one message.

```
What you send:
{"type":"Chat","text":"hello"}\n
What TCP may deliver:
Chunk 1: {"type":"Chat","te
Chunk 2: xt":"hello"}\n{"type":"Chat","text":"second"}\n
Your application must reconstruct complete lines.
```

```
// src/shared/jsonLines.ts
import type { Socket } from "node:net";

export async function* readLines(socket: Socket): AsyncGenerator<string> {
  let buffer = "";

  for await (const chunk of socket) {
    buffer += chunk.toString("utf8");

    let newlineIndex = buffer.indexOf("\n");

    while (newlineIndex >= 0) {
      const line = buffer.slice(0, newlineIndex).trimEnd();
      buffer = buffer.slice(newlineIndex + 1);

      if (line.length > 0) {
        yield line;
      }

      newlineIndex = buffer.indexOf("\n");
    }
  }
}

export class LineReader {
  private readonly iterator: AsyncIterator<string>;

  public constructor(socket: Socket) {
    this.iterator = readLines(socket)[Symbol.asyncIterator]();
  }

  public async readLine(): Promise<string | undefined> {
    const result = await this.iterator.next();
    return result.done ? undefined : result.value;
  }
}
```

```

public async *lines(): AsyncGenerator<string> {
  while (true) {
    const line = await this.readLine();
    if (line === undefined) return;
    yield line;
  }
}

export function writeJsonLine(socket: Socket, value: unknown): void {
  socket.write(JSON.stringify(value) + "\n");
}

```

Memory rule: `socket.write()` sends bytes. `readLines()` creates messages. TCP itself does not know JSON, ChatMessage, LoginRequest or newline framing.

6. Recreate ClientConnection

```

// src/server/clientConnection.ts
import type { Socket } from "node:net";
import { LineReader, writeJsonLine } from "../shared/jsonLines.js";

export class ClientConnection {
  public userName: string | undefined;
  private readonly lineReader: LineReader;

  public constructor(private readonly socket: Socket) {
    this.lineReader = new LineReader(socket);
  }

  public lines(): AsyncGenerator<string> {
    return this.lineReader.lines();
  }

  public send(value: unknown): void {
    writeJsonLine(this.socket, value);
  }

  public close(): void {
    this.socket.end();
  }

  public destroy(): void {
    this.socket.destroy();
  }
}

```

C# ClientConnection	TypeScript ClientConnection
private TcpClient _client	private readonly socket: Socket
StreamReader + StreamWriter	Socket + readLines()/writeJsonLine()
public string? UserName	userName: string undefined
SendMessageAsync()	send() using socket.write()
Close()	socket.end() / socket.destroy()

Why is `userName` optional? A TCP connection exists before authentication succeeds. That is a real state in the program, so the type should represent it. After successful login/register, the server assigns `connection.userName = request.userName`.

7. Translate PasswordHasher: PBKDF2, Salt and Constant-Time Comparison

```
// src/server/passwordHasher.ts
import { pbkdf2, randomBytes, timingSafeEqual } from "node:crypto";
import { promisify } from "node:util";

const pbkdf2Async = promisify(pbkdf2);
const iterations = 210_000;
const keyLength = 32;
const digest = "sha256";

export interface PasswordRecord {
  salt: string;
  passwordHash: string;
}

export async function hashPassword(password: string): Promise<PasswordRecord> {
  const salt = randomBytes(16);
  const hash = await pbkdf2Async(password, salt, iterations, keyLength, digest);

  return {
    salt: salt.toString("base64"),
    passwordHash: hash.toString("base64")
  };
}

export async function verifyPassword(
  password: string,
  saltBase64: string,
  expectedHashBase64: string
): Promise<boolean> {
  const salt = Buffer.from(saltBase64, "base64");
  const expected = Buffer.from(expectedHashBase64, "base64");
  const actual = await pbkdf2Async(password, salt, iterations, keyLength, digest);

  return actual.length === expected.length && timingSafeEqual(actual, expected);
}
```

C# equivalent concepts: RandomNumberGenerator -> randomBytes; Rfc2898DeriveBytes/PBKDF2 -> pbkdf2; CryptographicOperations.FixedTimeEquals -> timingSafeEqual; byte[] -> Buffer.

Security rule: Store only salt + derived password hash. Never put plaintext passwords in users.json. Keep the asynchronous PBKDF2 version so hashing does not block the Node event loop while clients are connected.

8. Users.json and UserStore

```
// src/server/userStore.ts
import { mkdir, readFile, writeFile } from "node:fs/promises";
import { dirname } from "node:path";

export interface UserAccount {
  userName: string;
  salt: string;
  passwordHash: string;
}

export class UserStore {
  public constructor(private readonly filePath: string) {}

  public async getAll(): Promise<UserAccount[]> {
    try {
      const json = await readFile(this.filePath, "utf8");
      return JSON.parse(json) as UserAccount[];
    }
  }
}
```

```

    } catch (error) {
      if (isNodeError(error) && error.code === "ENOENT") {
        return [];
      }
      throw error;
    }
  }

  public async saveAll(users: UserAccount[]): Promise<void> {
    await mkdir(dirname(this.filePath), { recursive: true });
    await writeFile(this.filePath, JSON.stringify(users, null, 2), "utf8");
  }

  function isNodeError(error: unknown): error is NodeJS.ErrnoException {
    return error instanceof Error;
  }

```

data/users.json - initial content

```
[ ]
```

Concurrency warning: A JSON file is fine for this learning project and two users, but `read -> modify -> write` is not a transactional database operation. Two registrations arriving at exactly the same time can race. Production code would use a database or serialize writes with a queue/mutex.

9. AuthenticationService: Login and Register

```

// src/server/authenticationService.ts
import type { AuthRequest, AuthResponse } from "../shared/messages.js";
import { hashPassword, verifyPassword } from "../passwordHasher.js";
import { UserStore } from "../userStore.js";

export class AuthenticationService {
  public constructor(private readonly userStore: UserStore) {}

  public async authenticate(request: AuthRequest): Promise<AuthResponse> {
    return request.type === "Register"
      ? this.register(request.userName, request.password)
      : this.login(request.userName, request.password);
  }

  private async register(userName: string, password: string): Promise<AuthResponse> {
    const users = await this.userStore.getAll();

    const exists = users.some(
      user => user.userName.toLowerCase() === userName.toLowerCase()
    );

    if (exists) {
      return { type: "AuthResponse", success: false, message: "Username already exists." };
    }

    const passwordRecord = await hashPassword(password);

    users.push({
      userName,
      salt: passwordRecord.salt,
      passwordHash: passwordRecord.passwordHash
    });

    await this.userStore.saveAll(users);
    return { type: "AuthResponse", success: true, message: "Registration successful." };
  }
}

```

```

private async login(userName: string, password: string): Promise<AuthResponse> {
  const users = await this.userStore.getAll();
  const user = users.find(
    item => item.userName.toLowerCase() === userName.toLowerCase()
  );

  if (!user) {
    return { type: "AuthResponse", success: false, message: "Invalid credentials." };
  }

  const valid = await verifyPassword(password, user.salt, user.passwordHash);

  return valid
    ? { type: "AuthResponse", success: true, message: "Login successful." }
    : { type: "AuthResponse", success: false, message: "Invalid credentials." };
}
}

```

This is almost the same responsibility as your C# AuthenticationService. The syntax changes more than the architecture. The important invariant is unchanged: the server authenticates before adding a connection to the active chat collection.

10. MessageDispatcher: Broadcast Without Mixing It into ChatServer

```

// src/server/messageDispatcher.ts
import type { ChatMessage } from "../shared/messages.js";
import { ClientConnection } from "../clientConnection.js";

export class MessageDispatcher {
  public broadcast(
    clients: ReadonlySet<ClientConnection>,
    message: ChatMessage
  ): void {
    for (const client of clients) {
      client.send(message);
    }
  }

  public chat(
    clients: ReadonlySet<ClientConnection>,
    name: string,
    text: string
  ): void {
    this.broadcast(clients, {
      type: "Chat",
      name,
      text,
      sentAt: new Date().toISOString()
    });
  }

  public joined(clients: ReadonlySet<ClientConnection>, name: string): void {
    this.broadcast(clients, {
      type: "Join",
      name,
      text: `${name} joined the chat.`,
      sentAt: new Date().toISOString()
    });
  }

  public left(clients: ReadonlySet<ClientConnection>, name: string): void {
    this.broadcast(clients, {
      type: "Leave",
      name,
      text: `${name} left the chat.`,
      sentAt: new Date().toISOString()
    });
  }
}

```

```

    }
  }
}

```

Why `ReadonlySet`?: The dispatcher needs to iterate clients, not own connection lifecycle. The type communicates that only ChatServer should add/remove sessions. This is the TypeScript equivalent of protecting ownership of a mutable collection.

11. ChatServer: Accept, Authenticate, Add, Broadcast, Remove

```

// src/server/chatServer.ts
import { createServer, type Server, type Socket } from "node:net";
import type { AuthRequest, ChatSendRequest } from "../shared/messages.js";
import { ClientConnection } from "../clientConnection.js";
import { AuthenticationService } from "../authenticationService.js";
import { MessageDispatcher } from "../messageDispatcher.js";

export class ChatServer {
  private readonly clients = new Set<ClientConnection>();
  private readonly server: Server;

  public constructor(
    private readonly authService: AuthenticationService,
    private readonly dispatcher: MessageDispatcher
  ) {
    this.server = createServer(socket => {
      void this.handleClient(socket);
    });
  }

  public start(port: number, host = "0.0.0.0"): void {
    this.server.listen(port, host, () => {
      console.log(`Server listening on ${host}:${port}`);
    });
  }

  private async handleClient(socket: Socket): Promise<void> {
    const connection = new ClientConnection(socket);
    let authenticated = false;

    try {
      for await (const line of connection.lines()) {
        const parsed: unknown = JSON.parse(line);

        if (!authenticated) {
          const request = this.asAuthRequest(parsed);
          if (!request) {
            connection.send({
              type: "AuthResponse",
              success: false,
              message: "Expected Login or Register request."
            });
            continue;
          }

          const response = await this.authService.authenticate(request);
          connection.send(response);

          if (!response.success) {
            continue;
          }

          connection.userName = request.userName;
          authenticated = true;
          this.clients.add(connection);
          this.dispatcher.joined(this.clients, request.userName);
        }
      }
    }
  }
}

```

```

        continue;
    }

    const chat = this.asChatSendRequest(parsed);
    if (!chat || !connection.userName) {
        continue;
    }

    this.dispatcher.chat(this.clients, connection.userName, chat.text);
} catch (error) {
    console.error("Client connection failed:", error);
} finally {
    if (this.clients.delete(connection) && connection.userName) {
        this.dispatcher.left(this.clients, connection.userName);
    }
    connection.destroy();
}
}

private asAuthRequest(value: unknown): AuthRequest | undefined {
    if (!value || typeof value !== "object") return undefined;
    const v = value as Record<string, unknown>;

    if (
        (v.type === "Login" || v.type === "Register") &&
        typeof v.userName === "string" &&
        typeof v.password === "string"
    ) {
        return v as unknown as AuthRequest;
    }

    return undefined;
}

private asChatSendRequest(value: unknown): ChatSendRequest | undefined {
    if (!value || typeof value !== "object") return undefined;
    const v = value as Record<string, unknown>;

    return v.type === "Chat" && typeof v.text === "string"
        ? (v as unknown as ChatSendRequest)
        : undefined;
}
}

```

Compare this to your C# server loop: accept connection -> wrap connection -> authenticate -> add to connected clients -> read chat messages -> dispatch -> remove on disconnect. The lifecycle is the same. The main Node-specific changes are `net.Server`, async iteration over the socket stream, and runtime shape checks after `JSON.parse()`.

Important: The `void this.handleClient(socket)` pattern intentionally starts an async client handler without awaiting it inside the synchronous connection callback. Each accepted socket gets its own asynchronous handler. Errors must therefore be caught inside `handleClient`.

12. Server Composition Root: main.ts

```

// src/server/main.ts
import { resolve } from "node:path";
import { AuthenticationService } from "../authenticationService.js";
import { ChatServer } from "../chatServer.js";
import { MessageDispatcher } from "../messageDispatcher.js";
import { UserStore } from "../userStore.js";

const userStore = new UserStore(resolve("data/users.json"));
const authService = new AuthenticationService(userStore);
const dispatcher = new MessageDispatcher();

```

```
const server = new ChatServer(authService, dispatcher);

server.start(5000);
```

Think of this as your manual DI container / Program.cs. Node does not provide ASP.NET Core dependency injection automatically. You construct dependencies explicitly and pass them into constructors. For a small TCP app, this is cleaner than adding a DI framework.

13. Client Console Input: Readline Instead of Console.ReadLineAsync

```
// src/client/consoleInput.ts
import { createInterface } from "node:readline/promises";
import { stdin as input, stdout as output } from "node:process";

export const consoleInput = createInterface({ input, output });
```

C#	Node/TypeScript
Console.WriteLine("Username: ") + Console.ReadLine()	await consoleInput.question("Username: ")
Console.ReadLineAsync()	readline/promises `question()`
Console.WriteLine()	console.log()

14. ChatClient: Connect, Authenticate, Receive and Send

```
// src/client/chatClient.ts
import { createConnection, type Socket } from "node:net";
import type { AuthRequest, AuthResponse, ChatMessage } from "../shared/messages.js";
import { LineReader, writeJsonLine } from "../shared/jsonLines.js";
import { consoleInput } from "../consoleInput.js";

export class ChatClient {
  private socket: Socket | undefined;

  public async start(host: string, port: number): Promise<void> {
    this.socket = await this.connect(host, port);
    const lineReader = new LineReader(this.socket);

    const authenticated = await this.authenticate(this.socket, lineReader);
    if (!authenticated) {
      this.socket.end();
      return;
    }

    const receiveTask = this.receiveLoop(lineReader);
    const sendTask = this.sendLoop(this.socket);

    await Promise.race([receiveTask, sendTask]);
    this.socket.end();
  }

  private connect(host: string, port: number): Promise<Socket> {
    return new Promise((resolve, reject) => {
      const socket = createConnection({ host, port });
      socket.once("connect", () => resolve(socket));
      socket.once("error", reject);
    });
  }

  private async authenticate(
    socket: Socket,
    lineReader: LineReader
  ): Promise<boolean> {
```

```

console.log("1. Login");
console.log("2. Register");

const choice = await consoleInput.question("Choice: ");
const userName = await consoleInput.question("Username: ");
const password = await consoleInput.question("Password: ");

const request: AuthRequest = {
  type: choice === "2" ? "Register" : "Login",
  userName,
  password
};

writeJsonLine(socket, request);

const line = await lineReader.readLine();
if (line === undefined) return false;

const response = JSON.parse(line) as AuthResponse;
console.log(response.message);
return response.success;
}

private async receiveLoop(lineReader: LineReader): Promise<void> {
  for await (const line of lineReader.lines()) {
    const message = JSON.parse(line) as ChatMessage;
    const time = new Date(message.sentAt).toLocaleTimeString();

    switch (message.type) {
      case "Join":
      case "Leave":
        console.log(`[${time}] *** ${message.text}`);
        break;
      case "Chat":
        console.log(`[${time}] ${message.name}: ${message.text}`);
        break;
    }
  }
}

private async sendLoop(socket: Socket): Promise<void> {
  while (!socket.destroyed) {
    const text = await consoleInput.question("");

    if (text.trim().toLowerCase() === "/quit") {
      return;
    }

    if (text.trim().length === 0) {
      continue;
    }

    writeJsonLine(socket, { type: "Chat", text });
  }
}

```

Subtle problem to notice: The client creates exactly one persistent `LineReader` for the socket. Authentication consumes one framed line from it, then the receive loop continues from the same reader. Avoid creating competing readers for one socket: whichever reader consumes a chunk first would steal bytes from the other.

15. Client Entry Point and Running Three Processes

```

// src/client/main.ts
import { ChatClient } from "../chatClient.js";

```

```
const client = new ChatClient();
await client.start("127.0.0.1", 5000);
```

Run it

```
# Terminal 1 - host/server
npm run server

# Terminal 2 - Alice
npm run client

# Terminal 3 - Bob
npm run client
```

1. Start the server first. It should print that it is listening on port 5000.
2. Start client 1 and choose Register. Register Alice.
3. Start client 2 and register Bob.
4. Both clients should receive Join messages after authentication.
5. Type a message in Alice. Bob and Alice should receive the broadcast.
6. Type /quit in Bob. Alice should receive the Leave message.
7. Stop and restart the server. users.json should still contain the registered accounts.

16. Debugging the TypeScript TCP Version

Symptom	Most likely cause	What to inspect
JSON.parse throws	You parsed a partial TCP chunk instead of a framed line	Confirm all reads go through readLines().
Two messages appear glued together	Missing newline framing or bad buffer splitting	Every write must append <code>`n`</code> .
Client connects but never authenticates	Server is waiting for a complete line	Log raw outgoing JSON + newline.
401-style login failure	Wrong password/hash/salt or username lookup	Inspect users.json; never log plaintext password.
Join works but chat does not	Parsed message type/text does not match union	Log parsed object and discriminant <code>`type`</code> .
One user disconnect crashes server	Unhandled socket/JSON exception escaped handler	Catch inside per-client handler/finally cleanup.
Duplicate user remains in list	Connection not deleted in finally	<code>`clients.delete(connection)`</code> must always run.
<code>`tsc --noEmit`</code> red lines	Type model and runtime states disagree	Fix types; do not silence with <code>`any`</code> .
Works on localhost, not another PC	Binding/firewall/network address issue	Server bind 0.0.0.0; client uses host LAN IP; allow port.

Debugging mindset: Separate protocol bugs from transport bugs. First prove bytes arrive. Then prove framing creates complete JSON lines. Then prove JSON parses. Then prove the ``type`` routes to the correct handler. Then prove business logic (auth/broadcast). This is the same layered stack-trace mindset you used in your C# API work.

17. Concurrency: C# Tasks vs the Node Event Loop

C# version: each connection often feels like an independent async Task. The .NET runtime schedules continuations and may use multiple threads depending on the operation.

Node version: JavaScript normally executes on one event-loop thread. Network IO is non-blocking, so one process can keep many sockets active without one OS thread per client. Async crypto/file operations may use libuv worker threads under the runtime, but your JS callbacks still re-enter through the event loop.

C# mental model	Node mental model
-----	-----
AcceptTcpClientAsync() +--> HandleClientAsync() +--> HandleClientAsync() +--> HandleClientAsync()	connection event / callback +--> async handleClient() +--> async handleClient() +--> async handleClient()
Many asynchronous operations can progress concurrently.	One JS event loop coordinating many non-blocking sockets.

Do not infer: "Single-threaded" does not mean "one client at a time". It means your JavaScript code is not normally executing two handlers simultaneously on two JS threads. IO completion is interleaved. You can still create logical race conditions around async read-modify-write operations such as Users.json registration.

18. Refactor After It Works: One Socket Reader, Explicit Client State

The working version already uses one persistent LineReader. After it works, take the next step: let one long-lived message pump own the socket read side and route messages according to an explicit client state. This scales better when you add more request/response message types.

```
type ClientState =
  | { kind: "Authenticating" }
  | { kind: "Chatting"; userName: string }
  | { kind: "Closed" };

// One receive loop owns the socket read side.
for await (const line of readLines(socket)) {
  const message: unknown = JSON.parse(line);

  if (state.kind === "Authenticating") {
    // Expect AuthResponse, resolve a pending auth Promise,
    // then transition state to Chatting.
    continue;
  }

  if (state.kind === "Chatting") {
    // Expect ChatMessage and render it.
  }
}
```

This is a state machine. Before authentication, the same TCP socket speaks an authentication protocol. After success, it speaks the chat protocol. Making that state explicit reduces "what type of message should I expect now?" bugs.

19. TypeScript Types Are Not Runtime Validation

```
// This compiles:
const response = JSON.parse(line) as AuthResponse;
console.log(response.success);

// But `as AuthResponse` does NOT verify the incoming JSON.
// A malicious/broken client could send:
{ "success": "banana" }
```

In C#, `JsonSerializer.Deserialize<AuthResponse>` maps into a runtime .NET type. TypeScript interfaces are erased when emitted to JavaScript. The `as` keyword is only a compiler assertion. The server-side helper methods in this guide therefore inspect `typeof`, `type` and required fields before treating untrusted JSON as a known message.`

Stage	Training version	Production direction
-------	------------------	----------------------

Parsing	JSON.parse + manual checks	Schema validator (for example Zod/Valibot) or explicit codecs
Protocol	newline-delimited JSON	Still valid; alternatively length-prefix / WebSocket depending needs
Users	users.json	PostgreSQL/SQLite with unique username constraint
Password hash	PBKDF2	PBKDF2/Argon2/scrypt with policy and migration strategy
Transport	plain TCP on local network	TLS or a secure tunnel when crossing untrusted networks

20. What to Test

Test	Why it matters
Register new user succeeds	Verifies store + hash + response.
Register duplicate username fails	Protects identity uniqueness.
Login correct password succeeds	Verifies PBKDF2 comparison.
Login wrong password fails	Negative auth path.
Unauthenticated Chat message is rejected/ignored	Authentication boundary.
Alice message reaches Bob	Core broadcast behavior.
Bob disconnect is removed	Prevents stale connections.
Join/Leave message type is correct	Protocol semantics.
Two JSON messages in one TCP chunk are separated	Framing correctness.
One JSON message split across chunks is reconstructed	Framing correctness.

Best first unit test: Test `readLines()` independently with an artificial stream/chunk sequence. Framing is the hidden foundation of every higher-level message. If framing is wrong, authentication and chat tests will fail for misleading reasons.

21. Recommended Build Order - Do Not Build Everything at Once

Milestone	Implement	Definition of done
1. Connect only	Server + one client	Client connects; server prints remote connection.
2. Raw lines	jsonLines.ts	Client sends one line; server reconstructs it.
3. JSON DTO	ChatSendRequest	Server parses <code>{type:"Chat", text:"..."}</code> .
4. Two clients	Set<ClientConnection>	Both can remain connected simultaneously.
5. Broadcast	MessageDispatcher	Alice text appears in Bob terminal.
6. Login	AuthRequest/AuthResponse	Only authenticated client enters chat.

7. Register	UserStore + PasswordHasher	New account survives restart.
8. Join/Leave	ChatMessage types	Both clients see session lifecycle.
9. /quit	Graceful client shutdown	Server removes connection and broadcasts Leave.
10. Harden	validation/errors/tests	Bad input does not crash host.

Why this order works: Every milestone isolates one unknown. If you jump directly to passwords + JSON + multi-client + persistence, a failed login could actually be a framing bug, a socket lifecycle bug, or a file race. Vertical slices make failures local.

22. Exercises That Force the C# -> TypeScript Translation to Stick

1. Before looking at the conversion table, write the TypeScript equivalent you expect for `TcpListener`, `TcpClient`, `Task`, `List<T>` and `JsonSerializer`.
2. Rewrite `ClientConnection` from memory. Then compare only the constructor, `send`, `lines`, and `close` responsibilities.
3. Explain in one sentence why `socket.on("data")` is not the equivalent of `ReadLineAsync()`.
4. Add a `Whisper` message type that includes `to: string`. Do not change the TCP framing layer.
5. Reject chat messages longer than 500 characters on the server, not only in the client.
6. Add a `/users` command that returns the currently connected usernames.
7. Replace `users.json` with SQLite or PostgreSQL after the JSON version works. Keep `AuthenticationService` unchanged as much as possible.
8. Create an integration test that starts a server on a test port, creates two Socket clients and proves Alice -> Bob broadcast.

Architecture test: If adding `/users` or `Whisper` forces you to edit `PasswordHasher` or `UserStore`, responsibilities have leaked. Protocol features should mainly affect message types, routing/dispatcher logic and the client display.

23. Side-by-Side Patterns You Should Memorize

```
C#
---
TcpClient client = new();
await client.ConnectAsync(host, port);

TypeScript
-----
const socket = createConnection({ host, port });
await new Promise<void>((resolve, reject) => {
  socket.once("connect", resolve);
  socket.once("error", reject);
});
```

```
C#
---
await writer.WriteLineAsync(JsonSerializer.Serialize(message));

TypeScript
-----
socket.write(JSON.stringify(message) + "\n");
```

```
C#
```

```

---
while ((line = await reader.ReadLineAsync()) is not null)
{
    // complete framed line
}

```

TypeScript

```

-----
for await (const line of readLines(socket)) {
    // complete framed line
}

```

C#

```

---
private readonly List<ClientConnection> _clients = [];
_clients.Add(client);
_clients.Remove(client);

```

TypeScript

```

-----
private readonly clients = new Set<ClientConnection>();
this.clients.add(client);
this.clients.delete(client);

```

C#

```

---
public async Task<AuthResponse> LoginAsync(LoginRequest request)

```

TypeScript

```

-----
public async login(request: AuthRequest): Promise<AuthResponse>

```

24. Do Not Translate C# Mechanically

Temptation	Why it is wrong / unnecessary
Create a NetworkStream wrapper because C# had one	Node Socket already is a Duplex stream.
Use classes for every DTO	Interfaces/types are usually clearer for plain JSON data.
Use `any` whenever TypeScript complains	You erase exactly the protection you are trying to learn.
Assume one data event = one JSON message	TCP is a byte stream; framing must be explicit.
Use synchronous PBKDF2/readFile calls	Blocking work freezes the event loop and delays all clients.
Add Express/Next.js	They solve HTTP/web concerns, not raw TCP. `node:net` is the right primitive here.
Copy ASP.NET DI patterns	Manual constructor injection is enough for this small Node program.
Trust `as AuthRequest` on network input	Type assertions do not validate runtime data.

25. After the Recreation Works

- Add server commands: /users, /whisper, /help.
- Add message IDs and acknowledgements so you can reason about delivery.
- Persist chat history in SQLite/PostgreSQL.
- Add rate limiting per connection.
- Add maximum line length to prevent a client from filling memory without sending a newline.
- Add TLS before using the protocol over an untrusted network.

- Replace console UI with React only after the TCP protocol is stable. A browser cannot open arbitrary raw TCP sockets directly, so a web UI would normally talk to a WebSocket/HTTP gateway rather than `node:net` from browser JavaScript.
- Containerize the host/server after local three-terminal testing is stable.

Good next architecture exercise: Keep the TypeScript TCP server unchanged and build a small WebSocket gateway that translates browser messages into your internal ChatMessage model. That demonstrates protocol adapters without throwing away the core domain/message design.

26. Knowledge Checklist

- ☐ I can explain why TCP is a stream and why newline framing exists.
- ☐ I know `net.Server` is the closest Node equivalent to TcpListener.
- ☐ I know `net.Socket` replaces the combined TcpClient/NetworkStream role.
- ☐ I can map Task<T> to Promise<T> without assuming Node uses the same threading model.
- ☐ I can explain why TypeScript interfaces disappear at runtime.
- ☐ I can use a discriminated union (`type`) to route messages.
- ☐ I can keep authentication before adding a client to the active chat set.
- ☐ I can hash passwords with PBKDF2 + random salt and compare with timingSafeEqual.
- ☐ I can explain why users.json is acceptable for training but not transactional persistence.
- ☐ I can debug transport -> framing -> parsing -> routing -> business logic in that order.

27. Official References

Node.js net module - TCP servers and sockets: <https://nodejs.org/api/net.html>

Node.js readline - promise-based console input: <https://nodejs.org/api/readline.html>

Node.js crypto - PBKDF2, randomBytes and timing-safe comparison: <https://nodejs.org/api/crypto.html>

TypeScript TSConfig reference - strict mode and Node module settings: <https://www.typescriptlang.org/tsconfig/>

Reference note: The architecture and examples in this guide are designed to preserve the responsibilities of your earlier C# MultiChat project while using Node.js/TypeScript primitives appropriately. Verify npm/runtime version requirements when you create the project, because Node and TypeScript continue to evolve.

28. The Mental Model to Keep



The important lesson is not that TypeScript has different method names. It is that the architecture survives the language change. TCP transport, message framing, authentication, session ownership, broadcasting and persistence are separate problems. Once you can identify those boundaries, moving between C#/.NET and TypeScript/Node becomes translation rather than starting over.